Notes

Data Modelling, Database, Data Warehouse & SQL

Dheeraj S Kulkarni

2025-2026

Data Modelling Concepts

Data modelling is the process of creating a conceptual, logical, and physical representation of data in a system. It defines how data is structured, stored, and accessed, and it forms the foundation for databases and data warehouses.

---

1. Stages of Data Models

a. Conceptual Data Model

- **Purpose**: Provides a high-level view of the system, focusing on business requirements rather than technical details.
- **Key Features**:
  - Defines entities, relationships, and attributes.
  - No primary/foreign keys or constraints.
- **Example**:
  - Entities: Customer, Order
  - Relationship: Customer places Order

b. Logical Data Model

- **Purpose**: Specifies the structure of data in detail, including relationships, attributes, and constraints, but is independent of the database technology.
- **Key Features**:
  - Defines primary keys, foreign keys, and data types.
  - Still independent of physical storage.
- **Example**:
  - Customer (CustomerID, Name, Email)
  - Order (OrderID, OrderDate, CustomerID)

c. Physical Data Model

- **Purpose**: Maps the logical model to the actual database implementation.
- **Key Features**:
  - Includes storage-specific details like indexing, partitions, and physical constraints.
  - Optimized for performance.
- **Example**:
  - SQL table definitions with data types, constraints, and indexes.

---

2. Core Data Modelling Concepts

a. Entities and Attributes

- **Entities**: Objects or concepts in the real world (e.g., Customer, Product).
- **Attributes**: Characteristics of entities (e.g., Customer has Name, Email).

b. Relationships

- Define how entities are related to each other.
- **Types**:
  - One-to-One (1:1): A Person has one Passport.
  - One-to-Many (1:N): A Customer places multiple Orders.
  - Many-to-Many (M:N): Students enroll in multiple Courses.

c. Keys

- **Primary Key (PK)**: Unique identifier for a record (e.g., CustomerID).
- **Foreign Key (FK)**: Attribute in one table referencing the primary key of another table.

d. Normalization

- The process of organizing data to reduce redundancy and improve consistency.
- **Forms**:
  - **1NF**: Eliminate duplicate columns and ensure atomicity.
  - **2NF**: Ensure all attributes depend on the whole primary key.
  - **3NF**: Eliminate transitive dependencies.

e. Denormalization

- Combining tables or reducing normalization for performance optimization.
- Used in data warehouses to simplify querying.

---

3. Data Modelling in Data Warehouses

a. Star Schema

- **Structure**:
  - Central **fact table** surrounded by **dimension tables**.
- **Advantages**:
  - Simple and intuitive.
  - Optimized for OLAP queries.
- **Example**:
  - Fact Table: Sales (SaleID, ProductID, Amount, DateID)
  - Dimensions: Product, Time, Customer, Region

b. Snowflake Schema

- **Structure**:
  - Dimensions are normalized into multiple related tables.
- **Advantages**:
  - Reduces redundancy and saves storage.
- **Disadvantages**:
  - More complex queries.
- **Example**:
  - A Product dimension is split into Product and Category.

c. Fact Tables

- **Purpose**: Store numeric measures (e.g., Revenue, Quantity).
- **Types**:
  - **Transactional Facts**: Individual transactions.
  - **Snapshot Facts**: Periodic snapshots (e.g., daily inventory).

- **Accumulating Facts**: Show lifecycle data (e.g., order processing).

d. Dimension Tables

- **Purpose**: Provide descriptive context to facts.
- **Example**:
  - Time: Date, Week, Quarter, Year
  - Customer: Name, Age, Region

e. Slowly Changing Dimensions (SCD)

- Handle historical changes in dimension data.
- **Types**: SCD Types 0, 1, 2, 3, 4, 6.

---

4. Advanced Data Modelling Concepts

a. Hierarchies

- Define relationships within dimensions for drill-down/roll-up analysis.
- **Example**:
  - Geography: Country > State > City
  - Time: Year > Quarter > Month > Day

b. Surrogate Keys

- Synthetic primary keys are used in place of natural keys to ensure uniqueness and improve performance.
- **Example**:
  - Surrogate key CustomerSK instead of natural key CustomerID.

c. Bridge Tables

- Used to resolve many-to-many relationships.
- **Example**:
  - A StudentCourse bridge table linking Students and Courses.

d. Metadata

- Data about the data model, providing context and rules for usage.
- Types:
  - **Technical Metadata**: Table structures, indexes, constraints.
  - **Business Metadata**: Descriptions of business terms and metrics.

Summary Table of Dimension Types:

Dimension Type	Key Use Case
Conformed Dimension	Shared consistency across fact tables
Slowly Changing Dimension	Capturing changes over time
Junk Dimension	Combining unrelated low-cardinality flags
Role-Playing Dimension	Reusing dimensions in multiple contexts
Degenerate Dimension	Tracking keys without descriptive attributes
Inferred Dimension	Placeholder records for incomplete data
Shrunken Dimension	Aggregation at summary levels
Hierarchical Dimension	Enabling drill-down/roll-up analysis

Slowly Changing Dimension (SCD)

- **Definition**: Dimensions that capture changes in data over time.
- **Types**:
  - **Type 0**: Fixed values.
  - **Type 1**: Overwrite changes.
  - **Type 2**: Add a new row for changes.
  - **Type 3**: Add a new column for changes.
  - **Type 4**: Use a historical table.
  - **Type 6**: Hybrid approach.

Choosing an SCD Type

- **Type 0**: When data must never change (e.g., original launch date).
- **Type 1**: When historical data isn't necessary (e.g., email addresses).
- **Type 2**: When complete history is essential (e.g., address changes).
- **Type 3**: When only the most recent change is relevant (e.g., product reclassification).
- **Type 4**: When separating current and historical data is practical (e.g., employee role history).
- **Type 6**: When combining flexibility and history is required (e.g., customer loyalty programs).

How Data Warehouse Differs from a Database

Feature	Database	Data Warehouse
Purpose	Designed for day-to-day operations (OLTP).	Optimized for analytics and reporting (OLAP).
Data Type	Current, transactional data.	Historical, aggregated data.
Schema	Highly normalized (to reduce redundancy).	Denormalized (to optimize query performance).
Read/Write Operations	Frequent inserts/updates/deletes.	Primarily read operations with periodic data loads.
Data Integration	Data from one system (or a few).	Consolidates data from multiple systems.
Performance	Optimized for fast transactions.	Optimized for complex queries and large-scale analytics.
Users	Operational users (e.g., sales clerks).	Analytical users (e.g., business analysts).
Updates	Real-time updates.	Periodic updates (batch or ETL processes).
Indexing	Primary and foreign keys.	Star or snowflake schema indexing for analytical queries.

Kimball Data Modeling vs Relational Data Modeling:

Aspect	Kimball Data Modelling	Relational Data Modelling
Purpose	Designed for data warehousing and analytics (OLAP).	Designed for transactional systems (OLTP).
Focus	Optimizes data for querying and reporting.	Ensures data integrity and minimizes redundancy.
Approach	Dimensional modelling with fact and dimension tables.	Entity-relationship (ER) modelling with entities and relationships.
Schema Type	Star schema or Snowflake schema.	Fully normalized schema (up to 3NF or higher).
Normalization	Denormalized for performance and simplicity.	Highly normalized to reduce redundancy.
Data Granularity	Stores aggregated and historical data.	Stores detailed, current, and operational data.
Scalability	Suited for analytical scalability and large datasets.	Scales well for transactional operations.
Fact Tables	Central table with measures (e.g., sales, revenue).	Not applicable.
Dimension Tables	Descriptive attributes (e.g., product, time, customer).	Not applicable.
Key Use Cases	Dashboards, reporting, and business intelligence.	Transactional systems and operational applications.

Design Philosophy	Data is organized for readability and reporting.	Data is organized for storage efficiency and integrity.
Example	- **Fact Table**: `Sales` with `SalesAmount`, `ProductID`, `DateID`. - **Dimension Table**: `Product` with `ProductName`, `Category`.	- **Table 1**: `Customer` with `CustomerID`, `Name`, `Email`. - **Table 2**: `Order` with `OrderID`, `CustomerID`, `OrderDate`.
When to Use	For analytical systems, data warehouses, and historical analysis.	For operational systems, applications, and transaction processing.

Types of Approaches in the Dimensional Data Modelling

1. Bottom-Up Approach (Kimball Methodology)

- **Description**: Focuses on creating **data marts** first for specific business processes (e.g., sales, marketing) and then integrating them into a data warehouse.
- **Characteristics**:
  - Prioritizes business needs and end-user reporting requirements.
  - Uses **star schema** or **snowflake schema**.
  - Scalable by adding data marts as needed.
- **Use Case**: Smaller organizations or teams focusing on immediate reporting needs.

---

2. Top-Down Approach (Inmon Methodology)

- **Description**: Starts with building a **centralized enterprise data warehouse** that stores normalized data, and then data marts are created for specific reporting purposes.
- **Characteristics**:
  - Enterprise-focused with a highly normalized data warehouse (3NF).
  - Data marts are derived from the central repository.
  - Emphasizes data consistency and integration.
- **Use Case**: Large organizations require a single source of truth for analytics.

---

3. Hybrid Approach

- **Description**: Combines the strengths of both the bottom-up and top-down approaches.
- **Characteristics**:
  - Develops a **centralized data warehouse** (top-down) while also building **data marts** (bottom-up) for quick wins.
  - Balances enterprise-wide consistency with flexibility for departmental needs.
- **Use Case**: Organizations aiming for scalability while addressing immediate reporting needs.

Easy way:

Bottom-up approach: इसका मतलब है कि आप पहले छोटे-छोटे हिस्से बनाते हैं—जैसे कि अलग-अलग डिपार्टमेंट के लिए छोटे डेटा मार्ट्स—और फिर उन सभी को मिलाकर एक बड़ा डेटा वेयरहाउस तैयार कर लेते हैं। यानी पहले छोटे-ब्लॉक्स, फिर पूरा घर।

Top-down approach: इसमें आप पहले एक मास्टर प्लान यानी एक सेंट्रल बड़ा डेटा वेयरहाउस बना लेते हैं, और उसके बाद ज़रूरत के हिसाब से उसमें से छोटे-छोटे डिपार्टमेंट या डेटा मार्ट्स निकालते हैं। यानि पहले पूरा नक्शा तैयार, फिर छोटे-छोटे कमरे।

Hybrid approach: यह दोनों का मिक्स है। कभी-कभी आप कुछ हिस्से नीचे से ऊपर की तरफ (bottom-up) और कुछ ऊपर से नीचे की तरफ (top-down) करके दोनों तरीकों का फायदा उठाते हैं।

Order of Execution of an SQL Query

1. `FROM` Clause
2. `WHERE` Clause
3. `GROUP BY` Clause
4. `HAVING` Clause
5. `SELECT` Clause
6. `ORDER BY` Clause
7. `LIMIT` Clause

How to Optimize SQL Queries Using the Order of Execution

- Use only the required tables.
- Avoid excessive joins and subqueries unless necessary.
- Filter data as early as possible to reduce the dataset size.
- Avoid Functions on Indexed Columns.
- Minimize the number of grouped columns.
- Use columns with low cardinality for grouping.
- Push as many filters as possible to the `WHERE` clause instead of `HAVING` to reduce intermediate data.
- Select only the required columns. Avoid `SELECT *`
- Use `LIMIT` to reduce the number of rows sorted if applicable.
- Use tools like **EXPLAIN** or **EXPLAIN ANALYZE** to check query execution plans and identify bottlenecks like table scans or improper index usage.
- Use `DISTINCT` judiciously. Ensure duplicate data is minimized at the source
- Precompute and store results for frequently used queries to avoid recomputation.
- Use `UNION ALL` instead of `UNION` if duplicate rows are not an issue, as `UNION` performs an extra step to remove duplicates.

Key Phrases for Interviews

- "I ensure indexing on columns used in JOINs, WHERE, and ORDER BY clauses."
- "I analyze query execution plans to identify bottlenecks."
- "I prioritize filtering data early in the WHERE clause to reduce processing overhead."
- "I avoid correlated subqueries and instead use joins or CTEs."
- "I optimize UNION queries by using UNION ALL where duplicates aren't a concern."

AWS vs Azure Services

Here's a comparison table of **AWS and Azure services** that serve similar purposes:

Category	AWS Service	Azure Service
Compute	Amazon EC2	Azure Virtual Machines (VMs)
Container Orchestration	Amazon Elastic Kubernetes Service (EKS)	Azure Kubernetes Service (AKS)
Serverless	AWS Lambda	Azure Functions
Storage	Amazon S3	Azure Blob Storage
File Storage	Amazon Elastic File System (EFS)	Azure Files
Block Storage	Amazon Elastic Block Store (EBS)	Azure Disk Storage
Networking	Amazon VPC	Azure Virtual Network (VNet)
Load Balancer	Elastic Load Balancing (ELB)	Azure Load Balancer
Content Delivery	Amazon CloudFront	Azure Content Delivery Network (CDN)
DNS Management	Amazon Route 53	Azure DNS
Databases	Amazon RDS	Azure SQL Database
	Amazon DynamoDB	Azure Cosmos DB
Data Warehouse	Amazon Redshift	Azure Synapse Analytics
Big Data	Amazon EMR	Azure HDInsight
Identity	AWS IAM	Azure Active Directory (AAD)
Monitoring	Amazon CloudWatch	Azure Monitor
IoT	AWS IoT Core	Azure IoT Hub
AI/ML	Amazon SageMaker	Azure Machine Learning

Queue Service	Amazon SQS	Azure Queue Storage
Pub/Sub	Amazon SNS	Azure Service Bus
Secrets Management	AWS Secrets Manager	Azure Key Vault
DevOps	AWS CodePipeline	Azure DevOps
Migration	AWS Migration Hub	Azure Migrate
Hybrid Cloud	AWS Outposts	Azure Stack
Backup	AWS Backup	Azure Backup

Open Table Formats Overview

Open Table Formats are modern **storage formats** designed to bring **ACID transactions, schema evolution, and performance optimizations** to **data lakes**. They provide **structured table management** on top of file-based storage (e.g., Parquet, ORC, Avro) and allow multiple processing engines like **Apache Spark, Trino, Presto, Flink, and Hive** to work seamlessly with large-scale datasets.

Choosing the Right Table Format

Use Case	Best Table Format
Large-scale **batch analytics** (data lakehouse)	**Apache Iceberg**
Unified **batch & streaming** workloads	**Delta Lake**
Real-time updates & CDC (Change Data Capture)	**Apache Hudi**
Time Travel & Versioning	**Iceberg & Delta**
Best for **Databricks users**	**Delta Lake**
Best for **AWS Athena, Presto, Trino**	**Apache Iceberg**

Conclusion

- **Apache Iceberg →** Best for **large-scale analytics** with hidden partitions & fast query performance.
- **Delta Lake →** Best for **Databricks/Spark** users who need **ACID transactions & time travel**.
- **Apache Hudi →** Best for **frequent updates, CDC, and real-time data ingestion**.

Major Differences: RDBMS vs. Data Warehouse vs. Apache Iceberg

- **RDBMS** is ideal for **real-time transactions** but struggles with big data and analytics.
- **Data Warehouses** optimise for **analytical workloads** but are expensive for large, unstructured datasets.
- **Apache Iceberg** is a **lakehouse format** that brings **data warehouse features** (ACID, schema evolution) to **data lakes**.

Data Warehouse Concepts

What is a Data Warehouse (DWH)?

A **Data Warehouse** is a **centralised repository** that stores data from multiple sources (databases, applications, flat files, APIs, etc.) in a **structured, integrated, and historical format**, optimised for **reporting and analytics** — not for daily transactions.

Key Characteristics (Inmon's definition)

1. **Subject-Oriented** → Organised around key business areas (Sales, Finance, HR)
2. **Integrated** → Brings together data from multiple heterogeneous sources.
3. **Non-volatile** → Data is stable; once loaded, not frequently changed (only new data added).
4. **Time-Variant** → Stores historical data for trend and time-based analysis.

Why Do We Need a DWH?

- To provide a **single source of truth** for business intelligence (BI).
- To support **decision-making, trend analysis, forecasting, and dashboards**.
- To integrate and clean messy operational data

Components of a Data Warehouse

- **ETL/ELT Processes** → Extract, Transform, Load data from sources.
- **Staging Area** → Temporary space for cleansing & transformation.
- **Data Warehouse Storage** → Central database (on-prem or cloud).
- **Data Marts** → Subsets of a data warehouse focused on specific business areas.
- **BI / Analytics Layer** → Reporting tools (Power BI, Tableau, Looker, etc.).

Examples of Modern Data Warehouses

- **On-premise:** Teradata, Oracle, IBM Netezza.
- **Cloud-based:** Snowflake, Google BigQuery, Amazon Redshift, Azure Synapse.

Data Lake vs Data Warehouse

Data Lake

- **Definition:** Central repository that stores **raw data** in its native format (structured, semi-structured, unstructured).
- **Purpose:** For **data scientists, engineers** — advanced analytics, machine learning, big data processing.
- **Schema: Schema-on-read →** apply structure only when reading data.
- **Storage Type:** Cheap, scalable object storage.
- **Examples:** Amazon S3 + Lake Formation, Azure Data Lake Storage (ADLS), Google Cloud BigLake, Hadoop HDFS.
- **Data Types:** Tables, JSON, CSV, images, videos, logs, IoT data.
- **Users:** Data Engineers, Data Scientists.
- **Analogy:** Like a **storage tank** — holds everything, clean or messy, until you decide how to use it.

Data Warehouse (DWH)

- **Definition:** Centralised repository for **processed, structured, and historical data** optimised for reporting & analytics.
- **Purpose:** For **business analysts, BI teams, executives** — dashboards, KPIs, decision-making.
- **Schema: Schema-on-write →** data structured before loading.
- **Storage Type:** Relational databases optimised for queries.

- **Examples:** Snowflake, BigQuery, Amazon Redshift, Azure Synapse, Teradata.
- **Data Types:** Structured/tabular (sales, finance, HR data).
- **Users:** Business Analysts, Decision-makers.
- **Analogy:** Like a **library** — only well-organized, categorized books (data) are stored for quick access.

Quick Difference Table

Feature	Data Lake	Data Warehouse
Data Type	Raw (structured, semi, unstructured)	Processed & structured
Schema	Schema-on-read	Schema-on-write
Purpose	Big data, ML, exploration	BI, reporting, dashboards
Users	Data Scientists, Engineers	Business Analysts, Executives
Cost	Cheaper (storage)	Costlier (compute + optimization)
Examples	S3, ADLS, GCS, Hadoop	Snowflake, Redshift, BigQuery

- **Data Lake** = *Store everything, decide later how to use it.*
- **Data Warehouse** = *Store only what's cleaned, structured, and ready for business use.*

OLTP and OLAP

OLTP (Online Transaction Processing)

- **Definition:** Systems designed for handling **day-to-day transactions** in real time.
  Focus: Fast insert, update, and delete.
- **Data Type:** Current, detailed, highly normalised (3NF).
- **Users:** Operational staff (bank tellers, e-commerce systems).
- **Examples:** Banking system, ATM transactions, flight booking systems, e-commerce checkout.

OLAP (Online Analytical Processing)

- **Definition:** Systems designed for **data analysis, reporting, and decision-making**.
- **Focus:** Complex queries, aggregations, trend analysis.
- **Data Type:** Historical, summarised, denormalised (star/snowflake schema).
- **Users:** Analysts, managers, executives.
- **Examples:** Sales trend analysis, quarterly financial reports, marketing campaign effectiveness, Power BI/ Tableau dashboards.

Quick Comparison Table

Feature	OLTP (Transactions)	OLAP (Analytics)
Purpose	Run day-to-day business ops	Support decision-making
Operations	Insert, Update, Delete, Select	Complex queries, Aggregations
Data	Current, real-time	Historical, summarised
DB Design	Normalised (3NF)	Denormalised (Star/Snowflake)

Query Speed	Milliseconds (fast for small)	Seconds/minutes (large queries)
Users	Clerks, end-users	Analysts, managers, executives
Examples	ATM, POS system, airline booking	Sales dashboard, KPI reports

---

One-line summary:

- **OLTP** = **"Run the business" (transactions).**
- **OLAP** = **"Analyse the business" (insights).**

ETL vs ELT

ETL (Extract → Transform → Load)

- **Process:**
  - **Extract** data from source systems (databases, APIs, flat files).
  - **Transform** data on a staging/ETL server (cleaning, joining, aggregating).
  - **Load** transformed data into the Data Warehouse.
- **When used:** Traditional data warehouses (limited compute/storage).
- **Examples:**
  - Extract customer data from MySQL → Transform using Informatica/Talend → Load into Oracle DWH.
  - Cleaning transactions before storing them in Teradata.
- **Schema:** Schema-on-write.

ELT (Extract → Load → Transform)

- **Process:**
  - **Extract** data from source systems.
  - **Load** raw data directly into the Data Warehouse/Data Lake.
  - **Transform** data inside the warehouse using its compute power (SQL/dbt/Snowflake/BigQuery).

- **When used:** Modern cloud data warehouses & lakehouses.
- **Examples:**
  - Extract IoT data → Load raw into Snowflake → Transform with SQL inside Snowflake.
  - Extract logs → Load into BigQuery → Transform with dbt queries.
- **Schema:** Schema-on-read (for lakes) or transformation inside the warehouse.

Quick Difference Table

Feature	ETL	ELT
Transform stage	Before loading (ETL tool)	After loading (inside DWH)
Best for	Traditional on-prem DWH	Modern cloud DWH / Data Lakes
Performance	Limited by the ETL server	Scales with warehouse compute
Latency	Higher (extra step)	Lower (direct load)
Examples	Informatica, Talend → Oracle	Snowflake, BigQuery, Redshift

- **ETL** = Old-school → Clean before storing.

- **ELT** = Modern → Store first, clean inside.

Dimensions vs Facts in Data Warehousing

Dimensions

- **What they are:** Descriptive attributes that provide **context** to facts.
- **Nature:** Textual, categorical, non-numeric (but can include IDs, codes, dates).
- **Examples:**
    - `Customer` (Name, Age, Gender, City)
    - `Product` (Name, Category, Brand)
    - `Time` (Date, Month, Year, Quarter)
    - `Store` (Location, Manager, Region)

Facts

- **What they are: Quantitative, measurable data** — numbers you aggregate and analyze.
- **Nature:** Numeric, additive/semi-additive.
- **Examples:**
    - `Sales Amount`
    - `Quantity Sold`
    - `Revenue`
    - `Profit`
    - `Website Clicks`

- **Dimensions = Nouns (who, what, where, when)**
- **Facts = Numbers (how much, how many)**

Example in context (Retail Store):

- **Fact Table:** Sales_Fact (Quantity = 100, Sales_Amount = $1,000)

- **Dimension Tables:**
    - Product_Dim (Product = “iPhone 15”, Category = “Mobile”)
    - Customer_Dim (Customer = “Alice”, City = “Dublin”)
    - Time_Dim (Date = 2025-08-29, Month = August)

Measures in a Data Warehouse (DWH)

- A **Measure** is a **numeric, quantitative value** stored in a **Fact Table** that can be aggregated (SUM, AVG, COUNT, MIN, MAX).
- They represent the **business performance metrics** you want to analyse.

Examples of Measures

- **Sales Domain:**
    - `Sales_Amount`
    - `Quantity_Sold`
    - `Discount`
    - `Profit_Margin`
- **Finance Domain:**
    - `Revenue`
    - `Cost`
    - `Operating_Expenses`
    - `EBITDA`
- **Retail/Marketing Domain:**
    - `Customer_Count`
    - `Website_Clicks`
    - `Conversion_Rate`

Types of Measures

1. **Additive Measures →** Can be summed across all dimensions.
    - Example: `Sales_Amount` across product, region, time.

2. **Semi-Additive Measures →** Can be aggregated along some dimensions, but not all.
    - Example: `Account_Balance` (valid across customers, but not across time).

3. **Non-Additive Measures →** Cannot be summed at all, only averaged or counted.
    - Example: `Profit_Margin %`, `Ratios`.

- **Measures = "Numbers you analyse" (How much? How many?)**
- **Dimensions = "Labels that describe those numbers" (Who? What? Where? When?)**

Grain (or Granularity)

The **Grain** (or Granularity) defines the **level of detail** at which data is stored in a fact table.

Examples of Grain

- **Sales Fact Table:**
    - At **Order Line level →** each row = 1 product in 1 order.
    - At the **Daily Store Sales level →** each row = total sales of a store for one day.

- **Web Analytics Fact Table:**

    - At the **Click level →** each row = one user click.
    - At the **Session level →** each row = one browsing session per user.

- **Finance Fact Table:**

    - At the **Transaction level →** each row = one transaction.

- At **the Monthly Snapshot level →** each row = account balance at month-end.

Why Grain Matters

- Determines **storage size** (fine grain = more rows, coarse grain = fewer rows).
- Affects **query performance**.
- Defines how **measures** (Sales, Revenue, Clicks, Balances) can be aggregated.

- **Fine-grain = zoomed-in photo (every detail, like each transaction).**
- **Coarse-grain = zoomed-out photo (summary, like monthly totals).**

Attributes in a Data Warehouse (DWH)

- **Attributes** are the **descriptive fields/columns** in **Dimension Tables** that provide **context** to measures in Fact Tables.
- They describe the **who, what, where, when, and how** around the numeric data.

Examples of Attributes

Customer Dimension

- Customer_ID
- Name
- Gender
- Age_Group
- City

Product Dimension

- Product_ID
- Product_Name
- Category
- Brand
- Size
- Color

Time Dimension

- Date_Key
- Day
- Month
- Quarter
- Year
- Holiday_Flag

Store Dimension

- Store_ID
- Store_Name
- Region
- Manager_Name

Types of Attributes

1. **Descriptive Attributes →** Describe the entity.
    - Example: Product_Name, Customer_Name.

2. **Operational/Status Attributes →** Current status.
    - Example: Customer_Active_Flag, Account_Status.

3. **Hierarchical Attributes →** Support drill-down/roll-up.
    - Example: Day → Month → Quarter → Year.

4. **Derived Attributes →** Calculated from other fields.
    - Example: Age (from DOB), Profit_Margin %.

Why Attributes Matter?

- They enable **filtering, grouping, and slicing & dicing** in OLAP analysis.
- Without attributes, facts (measures) would be just numbers with no meaning.

- **Facts = Numbers (sales, revenue, profit).**
- **Attributes = Labels/Details (which product, which customer, which date).**

Fact vs Dimension vs Attribute

Comparison Table

Aspect	Fact	Dimension	Attribute
Definition	Numeric, measurable data (metrics).	Descriptive entities that give context to facts.	Columns/fields inside dimension tables that describe the entity in detail.
Nature	Quantitative (numbers).	Qualitative (categories, descriptions).	Descriptive properties (text, codes, flags, hierarchies).

Examples	Sales_Amount, Quantity_Sold, Profit, Clicks.	Customer, Product, Time, Store.	Customer_Name, Gender, City, Product_Category, Month, Region.
Grain	Lowest level of measurement (transaction, daily total, etc.).	Level of analysis (who, what, where, when).	Details within dimensions (e.g., Customer Age, Product Colour).
Usage in BI	Used for aggregations (SUM, AVG, COUNT).	Used for slicing & dicing reports.	Used for filtering, grouping, drill-down.
Storage	Stored in Fact Tables.	Stored in Dimension Tables.	Stored as columns in Dimension Tables.

- **Fact** = **Numbers** (How much/many)
- **Dimension** = **Entities** (Who/What/Where/When)
- **Attribute** = **Properties** (Details of those entities)

Types of Dimensions

1. Conformed Dimension

- Shared across multiple fact tables or data marts.
- Example: `Customer_Dim` is used in both *Sales* and *Returns*.

2. Slowly Changing Dimension (SCD)

- Handles changes in dimension attributes over time.
- Types:
  - **Type 1:** Overwrite old data (e.g., update customer address).
  - **Type 2:** Keep history with new rows (e.g., track customers moving cities).
  - **Type 3:** Limited history with new columns (e.g., previous vs current address).

3. Junk Dimension

- Groups unrelated attributes into one dimension to avoid clutter.
- Example: `Flags_Dim` for promo_flag, return_flag, tax_flag.

4. Degenerate Dimension (DD)

- Dimension key stored in the fact table, but has no attributes.
- Example: `Order_Number` in the Sales fact table.

5. Role-Playing Dimension

- A single dimension is used multiple times in different roles.
- Example: `Date_Dim` used as Order_Date, Ship_Date, Delivery_Date.

6. Shrunken Dimension

- Subset of a larger dimension, usually for aggregated fact tables.
- Example: `Month_Dim` as a subset of `Date_Dim`.

7. Outrigger Dimension

- A dimension table linked to another dimension (not directly to a fact).
- Example: `Customer_Dim` linked to `Geography_Dim`.

8. Inferred Dimension

- Placeholder row created when fact arrives before complete dimension data.
- Example: Creating a dummy customer record when only Customer_ID is known at sale time.

- **Dimensions are like adjectives/nouns** describing facts, and these types are just different ways of handling business reality (history, shared use, incomplete info, etc.).

Types of Fact Tables

Three main types of Fact Tables in Data Warehousing:

- Transaction Fact Table
- Periodic Snapshot Fact Table
- Accumulating Snapshot Fact Table

---

1. Transactional Fact Table

- **Definition:**
  Stores data at the **lowest grain**, capturing **individual business events/transactions**.
- **Grain:** One row per transaction/event.
- **Nature:** Very detailed, grows rapidly.
- **Examples:**
  - Retail → Each row = one product in an order (`Order_ID`, `Product_ID`, `Quantity`, `Price`).
  - Banking → Each row = one ATM withdrawal or deposit.
  - E-commerce → Each row = one click or cart addition.

---

2. Periodic Snapshot Fact Table

- **Definition:**
  Stores **summarised facts** for a **specific, consistent time period** (daily, weekly, monthly, etc.).
- **Grain:** One row per entity **per period**.
- **Nature:** Less detailed, used for trend analysis.
- **Examples:**
  - Retail → Daily sales per store (`Store_ID`, `Date`, `Total_Sales`, `Total_Orders`).
  - Banking → Month-end balance for each account.
  - HR → Monthly employee headcount by department.

3. Accumulating Snapshot Fact Table

- Captures the **lifecycle of a process** that has a **definite beginning and end**.
- Updates rows as the process moves through different stages (instead of inserting new ones each time).
- One row per **business process instance** (e.g., one order, one claim, one loan application).

Key Features

- Contains **multiple date fields** for each milestone/stage.
- Measures (like processing time, costs) can be updated as the process progresses.
- Useful for **workflow tracking & SLA monitoring**.

Examples

- **Order Fulfillment**
  - A row is created when an order is placed.
  - Fields: `Order_Date`, `Payment_Date`, `Shipping_Date`, `Delivery_Date`.
  - Row gets updated as each stage is completed.

- **Loan Application**
  - Fields: `Application_Date`, `Approval_Date`, `Disbursement_Date`.

- **Insurance Claim**
  - Fields: `Claim_Submit_Date`, `Assessment_Date`, `Settlement_Date`.

Table Example (Order Process)

Order_ID	Order_Date	Payment_Date	Shipping_Date	Delivery_Date	Total_Amount
5001	2025-08-01	2025-08-02	2025-08-03	2025-08-05	1200

One row = the entire order's journey through all stages.

Comparison with Other Fact Tables

- **Transactional Fact →** Every single event (new row for each order line).
- **Periodic Snapshot Fact →** Summary at fixed intervals (daily sales per store).
- **Accumulating Snapshot Fact →** One row per process, updated over time.

Accumulating Snapshot Fact Table = **Tracking sheet** — one row per order/application, updated as it moves stage by stage.

Factless Fact Table (FFT)

Definition

A **Factless Fact Table** is a fact table that has **no numeric measures** — it only contains **foreign keys to dimensions** to capture the **occurrence of an event** or the **existence of a condition**.

Types of Factless Fact Tables

1. **Event Tracking FFT →** Captures that something *happened*.
    - Example: A Student attended a class, an employee logged into a system, patient visited the hospital.
    - A fact table just records the event (who, what, when, where), with no measure like sales amount.
2. **Coverage FFT →** Captures that something *did not happen* or *could happen*.
    - Example: Which products were on promotion (even if not sold), which students enrolled in which course (even if they didn't attend yet).

Examples

Event Tracking Example:

Date_Key	Student_Key	Course_Key	Instructor_Key
20250829	101	SQL101	501

Meaning: Student **101** attended **SQL101** class on 29-Aug-2025 with Instructor 501.
👉 No measures, but the fact itself = attendance event.

Coverage Example:

Date_Key	Product_Key	Promo_Key
20250829	202	PRM01

Meaning: On 29-Aug-2025, Product 202 was under Promotion PRM01 (even if no sales occurred).

Factless Fact Table = **Attendance register** — you mark presence/absence (event/coverage), no numbers involved.

Early Arriving Facts (a.k.a. Late Arriving Dimensions)

- **Definition:**
  When a **fact record arrives before its corresponding dimension record** exists in the warehouse.

  - Example: A sales transaction fact arrives with `Customer_ID = 123`, but the `Customer_Dim` table doesn't yet have that customer's details (maybe due to ETL lag or delay from the source system)

Problem

- The fact table has a **foreign key** pointing to a dimension row, but that row is missing.
- This breaks **referential integrity** and makes reports inconsistent.

Solutions

1. **Inferred Dimension Member (best practice)**

   - Create a **placeholder row** in the dimension with minimal info (e.g., only `Customer_ID`, default values for other attributes).
   - Later, update the row when the full dimension data arrives.

- Example: Insert row → `Customer_ID=123, Name="Unknown", City="Unknown"`.

2. **Staging the Fact until Dimension Arrives**
   - Temporarily hold fact rows in a staging area until corresponding dimension records are loaded.
3. **Retry / Delayed Load**
   - Keep retrying the fact load later after the dimensions are updated.

Example in Retail (Sales Fact + Customer Dim)

Sales_Fact (fact arrives first):

Date_Key	Customer_Key	Product_Key	Sales_Amount
2025-08-29	123	P001	500

Customer_Dim (not yet available):

Customer_Key	Name	City
Missing	*N/A*	*N/A*

Solution: Create inferred Customer row → (123, "Unknown", "Unknown").

High Date and Low Date

High Date

- **Definition:** A placeholder **maximum date** used to mark that a record is **currently valid** (no known end date).
  Common values: `9999-12-31`, `2999-01-01`, `4712-12-31` (Oracle default).

Customer_ID	City	Start_Date	End_Date	Current_Flag
101	Dublin	2022-01-01	2024-05-01	N
101	Cork	2024-05-02	**9999-12-31**	Y

Low Date

- **Definition:** A placeholder **minimum date** used to mark the **earliest possible start date** (or unknown start).
- **Common values:** `1900-01-01`, `0001-01-01`, `1970-01-01` (Unix epoch).

Customer_ID	City	Start_Date	End_Date
101	Dublin	**1900-01-01**	2024-05-01

Usage

- Both are common in **Slowly Changing Dimensions (SCD Type 2)** to handle **validity ranges** for historical records.
- Prevents NULL values in date fields (which can complicate queries).

- **Low Date = Beginning of time**
- **High Date = End of time (infinity)**